

Sri Lanka Institute of Information Technology



IT3012 : Intelligent Agents - Lab 4

Y3.S1.WD.AI.01.01

Malabe Campus

Student ID -	IT24100140
Student Name -	Prathiban HTP
Module Name -	Intelligent Agents
Module Number -	IT3012
LabSheet Number -	4
LabSheet Date -	2026/Aug/17

Task 1

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

```
171     def manhattan_distance(self, pos, goal):
172         x1, y1 = pos
173         x2, y2 = goal
174         return abs(x1 - x2) + abs(y1 - y2)
175
176     def euclidean_distance(self, pos, goal):
177         x1, y1 = pos
178         x2, y2 = goal
179         return math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
180
```

```
223
224
225 if __name__ == "__main__":
226     agent = SearchAgent()
227     print("Manhattan:", agent.manhattan_distance((0, 0), (3, 4)))
228     print("Euclidean:", agent.euclidean_distance((0, 0), (3, 4)))
229
```

```
Εισήγηση: 2'0
Μηνύματα: 3
C:\Users\user\AppData\Local\Microsoft\Windows\CurrentVersion\Explorer\History\History.dat
```

Task 2

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type="manhattan")`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

```

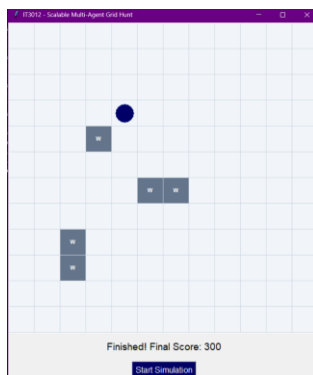
181 def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type="manhattan"):
182     """Returns the lowest estimated cost path from start to goal using A*"""
183     start = tuple(start_pos)
184     goal = tuple(goal_pos)
185     wall_set = set(walls)
186     reached_states = set()
187
188     h_cost = self._heuristic(start, goal, heuristic_type)
189     frontier = [(h_cost, 0, start, [])]
190
191     while frontier:
192         f_cost, g_cost, current_pos, path_taken = heapq.heappop(frontier)
193
194         if current_pos == goal:
195             return path_taken
196
197         if current_pos in reached_states:
198             continue
199
200         reached_states.add(current_pos)
201
202         for action, next_pos in self._successors(current_pos, wall_set, grid_size):
203             if next_pos not in reached_states:
204                 new_g_cost = g_cost + 1
205                 new_h_cost = self._heuristic(next_pos, goal, heuristic_type)
206                 new_f_cost = new_g_cost + new_h_cost
207                 heapq.heappush(frontier, (new_f_cost, new_g_cost, next_pos, path_taken + [action]))
208
209     return None
210
211 def _heuristic(self, pos, goal, heuristic_type):

```

```

87 def __init__(self):
88     self.plan = []
89     self.active_algo = 'ASTAR' #choose from BFS, DFS, UCS, ASTAR
90

```



Task 3

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

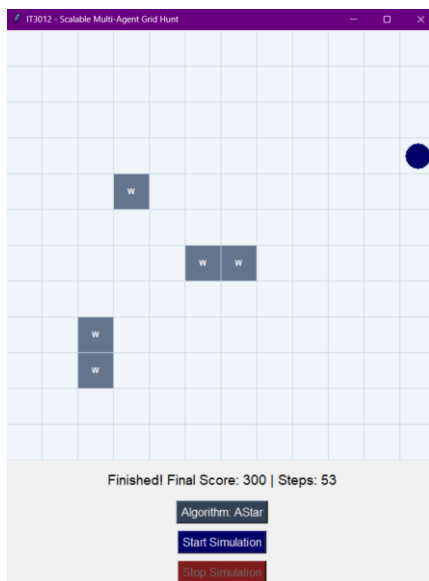
/courseweb.sliit.lk/mod/resource/view.php?id=486693

6, 9:23 PM

IT3012 - Practical 04: Informed Search

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

```
239 def _run_active_search(self, start_pos, goal_pos, walls, grid_size):
240     active_algo = self.active_algo.upper()
241
242     if active_algo == 'DFS':
243         return self.dfs_search(start_pos, goal_pos, walls, grid_size)
244     if active_algo == 'UCS':
245         return self.ucs_search(start_pos, goal_pos, walls, grid_size)
246     if active_algo == 'ASTAR':
247         return self.astar_search(start_pos, goal_pos, walls, grid_size)
248     return self.bfs_search(start_pos, goal_pos, walls, grid_size)
249
```



Theoretical Evaluation

1.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search selects the node with the lowest path cost from the start, which is represented by $g(n)$. A* Search selects the node using $f(n) = g(n) + h(n)$, where $g(n)$ is the cost already travelled and $h(n)$ is the estimated cost from the current node to the goal. Therefore, the main difference is that UCS only considers the cost travelled so far, while A* also considers an estimate of the remaining distance.

2.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is admissible in a 4-way grid because the agent can only move up, down, left, or right, so the heuristic does not overestimate the minimum number of moves needed to reach the goal. If a heuristic is not admissible and overestimates the actual remaining cost, A* may choose a non-optimal path and therefore lose its guarantee of finding the best solution.

3.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

Manhattan Distance is not suitable when diagonal movement is allowed because it can overestimate the actual shortest path. For example, moving from (0,0) to (3,3) has a Manhattan Distance of 6, but with diagonal movement the goal can be reached in only 3 moves. In this case, a heuristic such as **Chebyshev Distance** is more suitable when straight and diagonal moves have the same cost

4.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

For a problem with multiple food items, a stronger heuristic should estimate the cost of collecting all remaining food instead of considering only the nearest food item. One useful approach is to calculate the distance from the agent to the nearest food and combine it with the estimated minimum cost of connecting all remaining food locations, such as using a **Minimum Spanning Tree (MST)**. This gives A* a better estimate of the total remaining cost and can reduce unnecessary node exploration.